

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Case Study

Course Code:	ITA05	Course Title:	Computer Vision
CO Assessed:	CO3 – Precision (accurate feature extraction & analysis) (BL3)	Assessment:	Assessment Tool 2 – Case Study
Weightage:	40% of CIA	Total Marks:	100
CO3 Statement:	Analyze and extract image features using detection and matching techniques for effective image representation and comparison. (BL4)		
Student Name:	K. Vijay Bhaskar Reddy	Reg. No.:	192425155
Section / Batch:	AIML/2024	Date:	10/08/2026

Code of Conduct

I K. Vijay Bhaskar Reddy(2162425155) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

K. Vijay Bhaskar Reddy

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly	
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts	
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning	
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided	
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand	

1. Problem Statement

An image contains both straight edges and textured regions. An edge detection algorithm is applied, but the system fails to identify important key points required for image matching. The problem occurs because edge detection identifies boundaries, whereas image matching requires distinctive and repeatable points that can be localized accurately.

2. Understanding of the Case

The image contains different types of visual information:

Edges – boundaries between regions with different intensity or color.

Corners – points where two or more edges intersect or where image intensity changes significantly in multiple directions.

Textures – repeated or complex local patterns.

Key points – distinctive locations that can be reliably detected and matched between images.

The system's failure is caused by using an edge detector for a task that requires localized feature points.

3. Why Edge Detection Is Insufficient

Problem 1 – Edges are not unique points

A long straight edge may contain hundreds of edge pixels. There may be no unique answer as to which pixel should be selected as the matching point.

Problem 2 – Poor localization for matching

An edge represents a one-dimensional structure. Movement along the edge produces relatively little change in the local appearance, making the exact position of a point along a straight edge ambiguous.

Problem 3 – Texture produces many edges

Textured regions may contain numerous small intensity changes. An edge detector can generate many edge responses, but these responses are not necessarily distinctive or stable key points.

Problem 4 – Edges do not provide strong 2D constraints

For matching, it is desirable to know a point's location in both the x and y directions. A corner provides strong intensity variation in multiple directions, whereas a straight edge does not.

4. Why Corner Detection Is Better

A corner is a point where image intensity changes significantly in more than one direction. If a corner moves slightly, its surrounding intensity pattern changes significantly. Therefore, corners are distinctive, localized, repeatable, and suitable for feature matching.

5. Mathematical Principle

Corner detection can be explained using the structure tensor (second-moment matrix):

$$M = \begin{bmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{bmatrix}$$

where I_x is the image gradient in the x-direction and I_y is the image gradient in the y-direction.

Region	λ_1	λ_2	Interpretation
Flat region	Small	Small	No significant variation
Straight edge	Large	Small	Variation mainly in one direction
Corner	Large	Large	Strong variation in two directions
Texture	Often large	Often large	Multiple local variations

6. Edge vs Corner Detection

Feature	Edge Detection	Corner Detection
Detects	Boundaries	Distinctive points
Output	Lines/curves	Individual key points
Localization	Along an edge	Precise 2D location
Uniqueness	Usually low	Usually high
Matching suitability	Limited	Excellent
Texture handling	Can produce many responses	Can identify distinctive points
2D intensity variation	Usually one direction	Two directions
Examples	Canny, Sobel, Prewitt	Harris, Shi-Tomasi
Typical applications	Segmentation, shape detection	Feature matching, tracking, stitching

7. Suitable Corner Detection Algorithms

Harris Corner Detector

The Harris Corner Detector identifies locations where intensity changes significantly in two directions. Its response is $R = \det(M) - k(\text{trace}(M))^2$. A large positive R indicates a likely corner.

Shi-Tomasi Corner Detector

Shi-Tomasi uses $R = \min(\lambda_1, \lambda_2)$. A point is considered a strong corner when both eigenvalues are large. It is widely used for feature tracking and key-point detection.

8. Proposed Solution

Instead of relying only on edge detection, the system should use a corner/feature detection stage after initial image preprocessing.

Recommended pipeline:

Input Image → Preprocessing → Edge Detection → Corner Detection → Key Point Selection → Feature Description → Feature Matching

9. Example

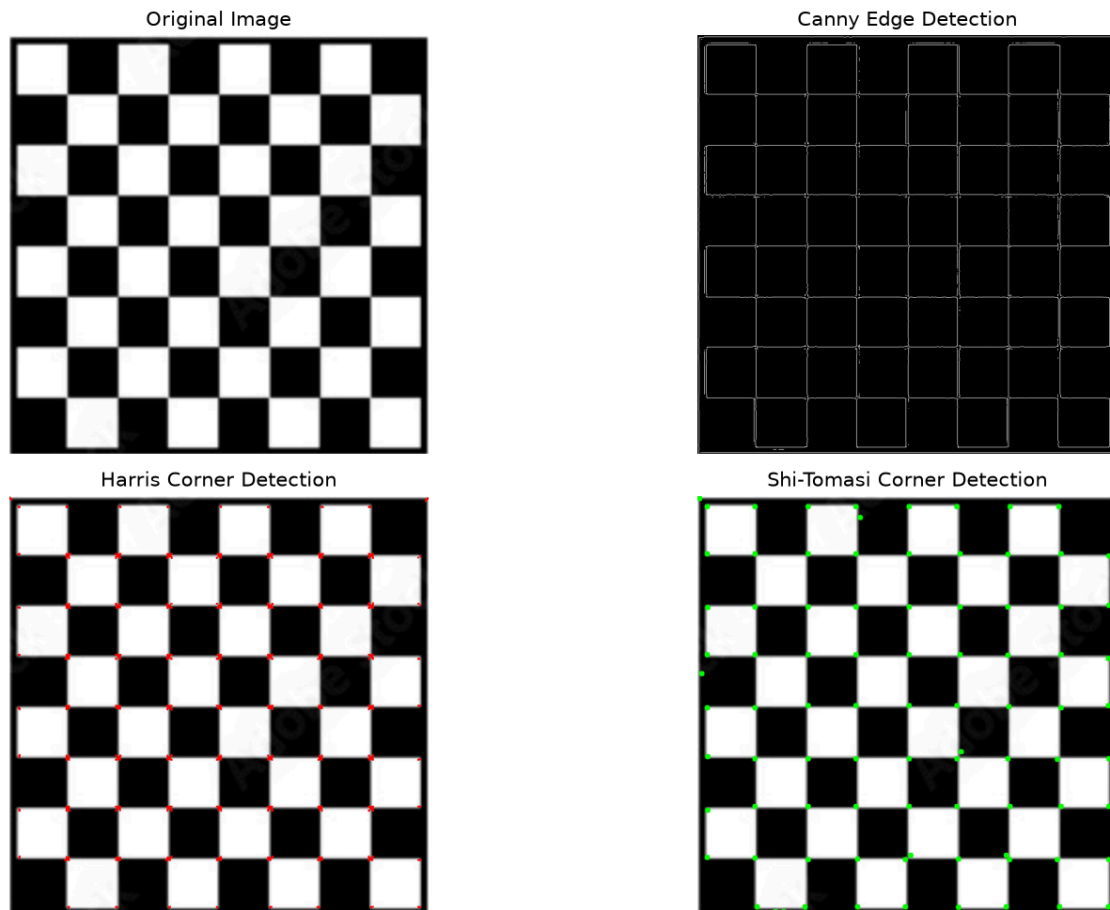
Program:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
img = cv2.imread("input.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
edges = cv2.Canny(gray, 100, 200)
harris = cv2.cornerHarris(
    np.float32(gray),
    2,          # block size
    3,          # Sobel kernel
    0.04        # Harris parameter
)
harris = cv2.dilate(harris, None)
harris_img = img.copy()
harris_img[harris > 0.01 * harris.max()] = [0, 0, 255]

corners = cv2.goodFeaturesToTrack(
    gray,
    100,        # maximum corners
    0.01,       # quality level
    10          # minimum distance
)
shi_img = img.copy()
if corners is not None:
    corners = np.int32(corners)

    for corner in corners:
        x, y = corner.ravel()
        cv2.circle(shi_img, (x, y), 5, (0, 255, 0), -1)

plt.figure(figsize=(14, 8))
plt.subplot(2, 2, 1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis("off")
plt.subplot(2, 2, 2)
plt.imshow(edges, cmap="gray")
plt.title("Canny Edge Detection")
plt.axis("off")
plt.subplot(2, 2, 3)
plt.imshow(cv2.cvtColor(harris_img, cv2.COLOR_BGR2RGB))
plt.title("Harris Corner Detection")
plt.axis("off")
plt.subplot(2, 2, 4)
plt.imshow(cv2.cvtColor(shi_img, cv2.COLOR_BGR2RGB))
plt.title("Shi-Tomasi Corner Detection")
plt.axis("off")
plt.tight_layout()
plt.show()
```



10. Analysis of the Failure

The original system fails because it treats edge pixels as equivalent to key points. An edge indicates a change in intensity along a boundary, while a useful key point should have a distinctive local neighborhood.

For reliable matching, an ideal key point should have:

- High local distinctiveness
- Accurate spatial localization
- Repeatability
- Good separability from neighboring points
- Resistance to small image transformations

Edge Detection → Boundary Information
Corner Detection → Distinctive Key Points

11. Improved Solution Design

A robust solution can combine both techniques:

Stage 1 – Edge Detection: Use Canny to identify major boundaries.

Stage 2 – Corner Detection: Use Harris or Shi-Tomasi to locate distinctive points.

Stage 3 – Feature Description: Generate descriptors around the detected key points.

Stage 4 – Matching: Compare descriptors between the two images.

12. Advantages of Corner Detection

Better localization – corners provide a specific (x, y) location.

Better matching – distinctive corner neighborhoods can be compared between images.

Better tracking – corners can be tracked across consecutive video frames.

Useful for image stitching – corresponding corners can help estimate transformations.

Useful for object recognition – distinctive points can help identify the same object under different viewpoints.

13. Applications

Image stitching

Computer vision

Autonomous vehicles

Augmented reality

Object tracking

Visual SLAM

Image registration

Object recognition

14. Final Justification

Edge detection alone does not provide sufficiently distinctive and localized features for reliable image matching. Straight edges generate many similar edge pixels, making their exact correspondence ambiguous. Textured regions can also produce numerous edge responses without providing stable key points.

Corner detectors such as Harris and Shi-Tomasi identify locations where intensity changes strongly in two directions. These points are more distinctive, spatially localized, and suitable for matching and tracking.

15. Conclusion

Edge detection should be used when the primary objective is to identify object boundaries, whereas corner/key-point detection should be used when the objective is feature matching, tracking, registration, or image stitching. For the given case, a combination of Canny edge detection, Harris/Shi-Tomasi corner detection, and feature descriptors provides a more robust and feasible solution.